Implementing Interactive Theorem Prover from Scratch

HIDEAKI TAKAHASHI, Columbia University, USA

TBD

1 PROPOSITIONS AS TYPES

A theorem prover has to decide whether a proof is correct. A proof written in English cannot be checked by a computer, and the Curry–Howard correspondence is exactly the concept that connects logic with programming: *propositions are types, and proofs are programs.* A proposition A is true if we can construct a term t of type A, written $t : A$.

For example, consider the proposition $A \to B$ ("A implies B"). To prove it, we assume A and construct a proof of B. This is exactly a function that takes a value of type A and returns a value of type B. Thus, logical implication corresponds to a function type. Here are three examples, where $\lambda x.\, e$ denotes a function taking x and returning e:

$$\begin{array}{rcll} P \to P & : & \lambda p.\, p & \text{Return the assumption.} \\ A \to B \to A & : & \lambda a.\, \lambda b.\, a & \text{Ignore the second argument.} \\ (A \to B) \to (B \to C) \to A \to C & : & \lambda f.\, \lambda g.\, \lambda a.\, g(f(a)) & \text{Compose two functions.} \end{array}$$

The same idea extends to universal quantification. To prove "for every $x : A$, $B(x)$ holds", we construct a function that takes any $x : A$ and returns a proof of $B(x)$. Unlike an ordinary function type $A \to B$, the return type $B(x)$ may depend on the input x. This is called a *dependent function type*, written

$$\Pi(x : A).\, B(x).$$

For example, "every proposition implies itself" is expressed as

$$\Pi(P{:}\text{Sort } 0).\, P \to P,$$

where Sort 0 is the type of propositions. Its proof is simply

$$\lambda(P{:}\text{Sort } 0).\, \lambda(p{:}P).\, p.$$

This gives us the central idea behind a theorem prover: *proving a theorem means constructing a term of the required type, and verifying a proof means type-checking that term.* The kernel implements these typing rules rather than reasoning about truth directly.

2 SYNTAX

We begin with a small dependent type theory consisting of five kinds of terms:

$$t, A, B ::= \text{Sort } u \mid x \mid \Pi(x{:}A).\, B \mid \lambda(x{:}A).\, t \mid t\ s$$

Here, u is a natural number and x is a variable name.

- Sort u: A universe containing types.
- x: A variable.
- $\Pi(x{:}A).\, B$: A Π-*type* (dependent function type), whose result type B may depend on x.
- $\lambda(x{:}A).\, t$: A λ-*abstraction*, i.e. a function that takes $x : A$ and returns t.
- $t\ s$: Function application.

Author's address: Hideaki Takahashi, ht2673@columbia.edu, Columbia University, USA.

When B does not depend on x, we write $A \to B$ instead of $\Pi(x{:}A).\, B$. Thus, ordinary function types are special cases of dependent function types. For example, the proposition "every proposition implies itself" and its proof are represented as

$$\underbrace{\lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(p{:}P).\, p}_{\text{proof}} \quad : \quad \underbrace{\Pi(P{:}\mathrm{Sort}\ 0).\, P \to P}_{\text{proposition = type of proof}}.$$

Notice that types and expressions share the same grammar. There is no separate syntax for types: whether a term is a valid type is determined by the type checker.

3 FREE VARIABLES AND SUBSTITUTION

Π and λ *bind* their variable: in $\lambda(x{:}A).\, t$, occurrences of x inside t refer to the argument. The binder's scope is the body only; x is not bound in the annotation A.

Definition 1 (Free variables).

$$\begin{aligned} \mathrm{FV}(\mathrm{Sort}\ u) &= \emptyset & \mathrm{FV}(\Pi(x{:}A).\, B) &= \mathrm{FV}(A) \cup (\mathrm{FV}(B) \setminus \{x\}) \\ \mathrm{FV}(x) &= \{x\} & \mathrm{FV}(\lambda(x{:}A).\, t) &= \mathrm{FV}(A) \cup (\mathrm{FV}(t) \setminus \{x\}) \\ \mathrm{FV}(t\ s) &= \mathrm{FV}(t) \cup \mathrm{FV}(s) \end{aligned}$$

The typing rule for application (§6) needs to replace a variable with a term. We write $t[x \mapsto s]$ for the result of replacing the free occurrences of x in t with s.

The only subtle case is a binder. Consider $(\lambda(y{:}A).\, x)[x \mapsto y]$. The body says "ignore the argument and return the outer x". Replacing x by y naively gives $\lambda(y{:}A).\, y$, the identity function, which means something else: the free y has been *captured* by the binder. Substitution must avoid this by renaming the binder first.

Definition 2 (Capture-avoiding substitution). *For $\square \in \{\Pi, \lambda\}$:*

$$\begin{aligned} x[x \mapsto s] &= s \\ y[x \mapsto s] &= y && (y \neq x) \\ \mathrm{Sort}\ u[x \mapsto s] &= \mathrm{Sort}\ u \\ (t_1\ t_2)[x \mapsto s] &= t_1[x \mapsto s]\ \ t_2[x \mapsto s] \\ (\square(x{:}A).\, B)[x \mapsto s] &= \square(x{:}A[x \mapsto s]).\, B && \textit{(we say that } x \textit{ is shadowed)} \\ (\square(y{:}A).\, B)[x \mapsto s] &= \square(y{:}A[x \mapsto s]).\, B[x \mapsto s] && (y \neq x,\ y \notin \mathrm{FV}(s)) \\ (\square(y{:}A).\, B)[x \mapsto s] &= \square(z{:}A[x \mapsto s]).\, (B[y \mapsto z])[x \mapsto s] && (y \neq x,\ y \in \mathrm{FV}(s),\ z \textit{ fresh}) \end{aligned}$$

Here "z fresh" means $z \notin \mathrm{FV}(s) \cup \mathrm{FV}(B) \cup \{x\}$.

4 ALPHA-EQUIVALENCE

The names of bound variables carry no meaning. For example, $\lambda(x{:}A).\, x$ and $\lambda(y{:}A).\, y$ are the same function, and $\Pi(P{:}\mathrm{Sort}\ 0).\, P \to P$ and $\Pi(Q{:}\mathrm{Sort}\ 0).\, Q \to Q$ are the same proposition. Terms that differ only in the names of bound variables are called *α-equivalent*.

Definition 3 (α-equivalence). *$=_\alpha$ is the smallest congruence on terms containing, for $\square \in \{\Pi, \lambda\}$,*

$$\square(x{:}A).\, B \ =_\alpha\ \square(y{:}A).\, B[x \mapsto y] \qquad \textit{whenever } y \notin \mathrm{FV}(B).$$

This definition is easy to state but not directly executable, since it does not say which names to choose. An implementation instead walks both terms in lockstep and remembers, for each side, the list of binders passed so far. A bound variable is then identified by *which* binder it refers to, not by its name. We write $E_1; E_2 \vdash s =_\alpha t$, where E_1 and E_2 are lists of names. Let $idx(E, x)$ be the position of the last occurrence of x in E, counted from the right starting at 0, or undefined if $x \notin E$. Taking the *last* occurrence handles shadowing: in $\lambda(x{:}A).\, \lambda(x{:}B).\, x$ the body refers to the inner x.

$$\frac{}{E_1; E_2 \vdash \mathrm{Sort}\ u =_\alpha \mathrm{Sort}\ u}\ (\alpha\text{-Sort}) \qquad \frac{idx(E_1, x) = idx(E_2, y) = i}{E_1; E_2 \vdash x =_\alpha y}\ (\alpha\text{-Bound}) \qquad \frac{x \notin E_1 \qquad x \notin E_2}{E_1; E_2 \vdash x =_\alpha x}\ (\alpha\text{-Free})$$

$$\frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash B =_\alpha B'}{E_1; E_2 \vdash \Pi(x{:}A).\, B =_\alpha \Pi(y{:}A').\, B'}\ (\alpha\text{-Pi}) \qquad \frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash t =_\alpha t'}{E_1; E_2 \vdash \lambda(x{:}A).\, t =_\alpha \lambda(y{:}A').\, t'}\ (\alpha\text{-Lam})$$

$$\frac{E_1; E_2 \vdash t_1 =_\alpha t_1' \qquad E_1; E_2 \vdash t_2 =_\alpha t_2'}{E_1; E_2 \vdash t_1\ t_2 =_\alpha t_1'\ t_2'}\ (\alpha\text{-App})$$

Fig. 1. Algorithmic α-equivalence. Two closed or open terms are α-equivalent iff $\cdot\,; \cdot \vdash s =_\alpha t$.

Free variables are compared by name (α-Free), and bound variables by position (α-Bound). A variable that is bound on one side and free on the other is never equal to anything, which is why there is no rule for that case. Because both sides descend through binders together, E_1 and E_2 always have the same length. So comparing positions counted from the right is the same as comparing positions counted from the left, and an implementation may store each list as a map from names to binding depths, provided it restores the previous entry when it leaves a binder. Using positions instead of names everywhere gives the *de Bruijn representation* [2], in which α-equivalent terms are syntactically identical.

5 REDUCTION AND CONVERSION

Applying a λ-abstraction computes by substitution:

$$(\lambda(x{:}A).\, t)\ s \ \longrightarrow_\beta \ t[x \mapsto s].$$

A subterm of the form $(\lambda(x{:}A).\, t)\ s$ is called a *β-redex*, and we also allow this step anywhere inside a term. A term with no β-redex is in *normal form*.

Computation can make two different-looking types equal. If $Id = \lambda(X{:}\mathrm{Sort}\ 0).\, X$, then a proof of $Id\ P$ should also be a proof of P, since $Id\ P \longrightarrow_\beta P$. This is captured by *definitional equality* (or *conversion*).

Definition 4 (Conversion). *$s \equiv t$ is the smallest equivalence relation that contains $=_\alpha$ and $\longrightarrow_\beta$ and is closed under all term constructors.*

To decide $s \equiv t$, the kernel normalizes both terms and compares the results with $=_\alpha$. This is correct because β-reduction is confluent: different reduction orders cannot lead to different normal forms. It also requires every well-typed term to *have* a normal form. That is true for the system in this section, but it is a deep theorem (strong normalization) [1]. A practical kernel also caps the number of reduction steps, so that a bug cannot make it loop forever.

6 TYPING RULES

Contexts and judgments. A *context* Γ is an ordered list of assumptions $x_1 : A_1, \ldots, x_n : A_n$. It records the type of each variable in scope, and we write $\mathrm{dom}(\Gamma)$ for the set $\{x_1, \ldots, x_n\}$. Since

C-Ext below forbids repeated names, Γ can be used like a dictionary from names to types, and T-Var is simply a lookup. Unlike a plain dictionary, however, the order matters: A_i may mention $x_1, \ldots, x_{i-1}$, as in $P : \mathrm{Sort}\ 0, h : P$. We use two judgments:

- $\vdash \Gamma$ ok: the context Γ is well formed.
- $\Gamma \vdash t : A$: under the assumptions in Γ, the term t has type A.

Read through Curry–Howard, $\Gamma \vdash t : A$ says that t is a proof of A from the hypotheses Γ.

$$\frac{}{\vdash \cdot \ \mathrm{ok}}\ (\text{C-Empty}) \qquad \frac{\vdash \Gamma \ \mathrm{ok} \qquad \Gamma \vdash A : \mathrm{Sort}\ u \qquad x \notin \mathrm{dom}(\Gamma)}{\vdash \Gamma, x : A \ \mathrm{ok}}\ (\text{C-Ext})$$

$$\frac{\vdash \Gamma \ \mathrm{ok}}{\Gamma \vdash \mathrm{Sort}\ u : \mathrm{Sort}\ (u+1)}\ (\text{T-Sort}) \qquad \frac{\vdash \Gamma \ \mathrm{ok} \qquad (x : A) \in \Gamma}{\Gamma \vdash x : A}\ (\text{T-Var})$$

$$\frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma, x : A \vdash B : \mathrm{Sort}\ v}{\Gamma \vdash \Pi(x{:}A).\, B : \mathrm{Sort}\ (\max(u, v))}\ (\text{T-Pi}) \qquad \frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x{:}A).\, t : \Pi(x{:}A).\, B}\ (\text{T-Lam})$$

$$\frac{\Gamma \vdash f : \Pi(x{:}A).\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[x \mapsto a]}\ (\text{T-App}) \qquad \frac{\Gamma \vdash t : A \qquad \Gamma \vdash B : \mathrm{Sort}\ u \qquad A \equiv B}{\Gamma \vdash t : B}\ (\text{T-Conv})$$

Fig. 2. Well-formed contexts and typing rules.

We read each rule from the bottom up, as a recipe for checking a term.

C-Empty, C-Ext. A context is built one assumption at a time, and each new assumption must have a type as its type. The side condition $x \notin \mathrm{dom}(\Gamma)$ forbids shadowing inside a context. Since we work up to α-equivalence, this is no restriction on terms: a binder that would shadow can always be renamed first. §10 shows why this condition matters.

T-Sort. Every universe lives in the next one; see §7 for why.

T-Var. A variable has the type it was assumed to have. Logically, this is "use a hypothesis".

T-Pi. $\Pi(x{:}A).\, B$ is a type if A is a type and B is a type once we assume $x : A$. It lives in the larger of the two universes (§7). Logically, this rule says when $A \to B$ and $\forall x{:}A.\, B$ are well-formed propositions.

T-Lam. To build a function from A to B, assume $x : A$ and build $t : B$. Logically, this is the introduction rule for $\to$ and $\forall$: to prove $A \to B$, assume A and prove B. Note that B may mention x, and it is kept as is in the resulting Π type.

T-App. If f maps each $x : A$ to an element of B, and $a : A$, then $f\ a$ has type B with x replaced by a. Logically, this is modus ponens when B does not mention x, and instantiation of a universal statement when it does. For example, if $f : \Pi(n{:}\mathrm{Nat}).\, n + 0 = n$, then $f\ 3 : 3 + 0 = 3$.

T-Conv. Types that are equal up to computation are interchangeable. This is the only rule that is not determined by the shape of t.

Why application matters. Without T-App, a hypothesis can be returned but never used. Roughly, the provable propositions are then those whose conclusion is literally one of the hypotheses, such as $P \to Q \to P$. Composition, $(P \to Q) \to (Q \to R) \to P \to R$, has the proof $\lambda(f{:}P \to Q).\, \lambda(g{:}Q \to R).\, \lambda(p{:}P).\, g\ (f\ p)$, which needs application.

7 UNIVERSES: WHY Sort u?

The typing rules of §6 are meant to be rules of logic. This section first makes that claim precise (§7.1). It then shows that the simplest choice of universes, a single universe that contains itself, breaks the claim (§7.3), and that the hierarchy Sort 0 : Sort 1 : ⋯ restores it (§7.4).

7.1 The Typing Rules as a Logic

Consider the logic whose formulas are built from proposition variables $P, Q, \ldots$ by implication and quantification over propositions:

$$\varphi, \psi ::= P \mid \varphi \to \psi \mid \forall P.\, \varphi$$

Its proofs are natural-deduction derivations of judgments $\Delta \vdash \varphi$, where Δ is a list of hypotheses. Each formula translates to a type, and each list of hypotheses to a context:

$$P^* = P, \qquad (\varphi \to \psi)^* = \varphi^* \to \psi^*, \qquad (\forall P.\, \varphi)^* = \Pi(P{:}\mathsf{Sort}\ 0).\, \varphi^*,$$

$$(\varphi_1, \ldots, \varphi_n)^* = P_1 : \mathsf{Sort}\ 0, \ldots, P_k : \mathsf{Sort}\ 0,\ h_1 : \varphi_1^*, \ldots, h_n : \varphi_n^*,$$

where $P_1, \ldots, P_k$ are the proposition variables free in Δ and $h_1, \ldots, h_n$ are fresh names for the hypotheses. Figure 3 places each logical rule next to the typing rule it corresponds to. Erasing the terms from the right column gives the left column exactly.

	Logic	**Type theory**
Hypothesis	$\dfrac{\varphi_i \in \Delta}{\Delta \vdash \varphi_i}$	$\dfrac{(h_i : \varphi_i^*) \in \Delta^*}{\Delta^* \vdash h_i : \varphi_i^*}$ (T-Var)
$\to$-intro	$\dfrac{\Delta, \varphi \vdash \psi}{\Delta \vdash \varphi \to \psi}$	$\dfrac{\Delta^*, h : \varphi^* \vdash t : \psi^*}{\Delta^* \vdash \lambda(h{:}\varphi^*).\, t : \varphi^* \to \psi^*}$ (T-Lam)
$\to$-elim	$\dfrac{\Delta \vdash \varphi \to \psi \qquad \Delta \vdash \varphi}{\Delta \vdash \psi}$	$\dfrac{\Delta^* \vdash f : \varphi^* \to \psi^* \qquad \Delta^* \vdash a : \varphi^*}{\Delta^* \vdash f\ a : \psi^*}$ (T-App)
$\forall$-intro	$\dfrac{\Delta \vdash \varphi \qquad P \notin \mathrm{FV}(\Delta)}{\Delta \vdash \forall P.\, \varphi}$	$\dfrac{\Delta^*, P : \mathsf{Sort}\ 0 \vdash t : \varphi^*}{\Delta^* \vdash \lambda(P{:}\mathsf{Sort}\ 0).\, t : \Pi(P{:}\mathsf{Sort}\ 0).\, \varphi^*}$ (T-Lam)
$\forall$-elim	$\dfrac{\Delta \vdash \forall P.\, \varphi \qquad \psi \text{ has no } \forall}{\Delta \vdash \varphi[P \mapsto \psi]}$	$\dfrac{\Delta^* \vdash f : \Pi(P{:}\mathsf{Sort}\ 0).\, \varphi^* \qquad \Delta^* \vdash \psi^* : \mathsf{Sort}\ 0}{\Delta^* \vdash f\ \psi^* : \varphi^*[P \mapsto \psi^*]}$ (T-App)

Fig. 3. Natural deduction (left) and the typing rules that implement it (right). Premises that only check that a type is well formed are omitted.

Two details in the figure deserve comment. The side condition $P \notin \mathrm{FV}(\Delta)$ of $\forall$-intro is what lets $P : \mathsf{Sort}\ 0$ be added after the hypotheses, since none of their types mention P. The side condition "ψ has no $\forall$" of $\forall$-elim comes from the premise $\psi^* : \mathsf{Sort}\ 0$: by T-Pi, $(\forall Q.\, \chi)^* = \Pi(Q{:}\mathsf{Sort}\ 0).\, \chi^*$ lives in Sort 1, not Sort 0. We return to this restriction in §7.4.

Theorem 1 (Propositions as types). *$\Delta \vdash \varphi$ is provable if and only if $\Delta^* \vdash t : \varphi^*$ for some term t.*

From left to right, the figure turns each step of a proof into a step of a typing derivation, so every proof becomes a term. From right to left, strong normalization (§11) lets us assume t is in normal

form, and a normal term of type φ^* is built only from variables, λ, and applications to terms or to types ψ^*, so it can be read back as a proof.

The theorem is what makes a type checker a proof checker. In particular, define $\bot = \forall P.\, P$ ("everything holds"). No proof of $\bot$ exists: if each P is read as true or false, every rule preserves truth under every reading, but $\forall P.\, P$ is false. By the theorem, no closed term has type

$$\bot^* = \Pi(P{:}\mathsf{Sort}\ 0).\, P.$$

This matters, because a closed $t : \bot^*$ would prove every proposition: $t\ C : C$ for every $C : \mathsf{Sort}\ 0$. A design for universes is acceptable only if it keeps this theorem true.

7.2 What Is the Type of Sort 0?

The $\forall$ rows of Figure 3 bind P with the annotation $\mathsf{Sort}\ 0$. T-Lam and T-Pi require every annotation to be a type, so $\mathsf{Sort}\ 0$ itself needs a type. Without one, a proposition variable could only be an assumption placed in the context from outside:

$$P : \mathsf{Sort}\ 0 \vdash \lambda(h{:}P).\, h : P \to P \qquad \text{versus} \qquad \vdash \lambda(P{:}\mathsf{Sort}\ 0).\, \lambda(h{:}P).\, h : \Pi(P{:}\mathsf{Sort}\ 0).\, P \to P.$$

The left judgment proves $P \to P$ for one fixed P. The right one is a single closed term that proves it for every P, and it can be applied later to any particular proposition. There are two natural answers to the question of what type $\mathsf{Sort}\ 0$ has. The first is "itself".

7.3 Type Is Not a Type

Suppose we replace all universes by a single universe Type that contains itself:

$$\frac{\vdash \Gamma\ \mathsf{ok}}{\Gamma \vdash \mathsf{Type} : \mathsf{Type}} \qquad \frac{\Gamma \vdash A : \mathsf{Type} \qquad \Gamma, x : A \vdash B : \mathsf{Type}}{\Gamma \vdash \Pi(x{:}A).\, B : \mathsf{Type}}$$

Translate $\forall P.\, \varphi$ as $\Pi(P{:}\mathsf{Type}).\, \varphi^*$. Every translated formula now has type Type, so the side condition "ψ has no $\forall$" disappears, which looks like a gain. But P now ranges over a collection that contains Type and every type built from it, including types that quantify over Type itself. This is the type-theoretic analogue of the set of all sets, and Russell's paradox can be replayed. Girard showed that the system then has a closed term

$$\vdash t_{\mathrm{G}} : \Pi(P{:}\mathsf{Type}).\, P,$$

and Hurkens later found a short one [3, 4]. So the theorem of §7.1 fails: $\bot$ has no proof, yet $\bot^*$ has a term, and a kernel built on these rules accepts a proof of every statement.

The damage reaches the algorithm as well. Subject reduction still holds, so if t_{G} had a normal form, it would be a normal closed term of type $\Pi(P{:}\mathsf{Type}).\, P$. The argument at the end of §11 shows that no such normal term exists. Hence t_{G} has no normal form, strong normalization fails, and Normalize in Algorithm 1 can run forever.

7.4 A Hierarchy of Universes

The second answer is that $\mathsf{Sort}\ 0$ lives in a larger universe, which lives in a larger one still:

$$\frac{\vdash \Gamma\ \mathsf{ok}}{\Gamma \vdash \mathsf{Sort}\ u : \mathsf{Sort}\ (u+1)}\ \text{(T-Sort)} \qquad \text{so} \qquad \mathsf{Sort}\ 0 : \mathsf{Sort}\ 1 : \mathsf{Sort}\ 2 : \cdots.$$

Combined with T-Pi, this rule prevents exactly the self-reference that the paradox needs. A type that quantifies over $\mathsf{Sort}\ u$ has the form $\Pi(x{:}\mathsf{Sort}\ u).\, B$, and T-Sort and T-Pi give it the type

$$\frac{\dfrac{}{\Gamma \vdash \mathsf{Sort}\ u : \mathsf{Sort}\ (u+1)}\ \text{T-Sort} \qquad \Gamma, x : \mathsf{Sort}\ u \vdash B : \mathsf{Sort}\ v}{\Gamma \vdash \Pi(x{:}\mathsf{Sort}\ u).\, B : \mathsf{Sort}\ (\max(u+1, v))}\ \text{T-Pi}$$

Since $\max(u+1, v) \geq u+1$, a type that quantifies over a universe is never a member of that universe. This property is called *predicativity*. With it, the system is consistent and strongly normalizing (§11), so the theorem of §7.1 holds.

Predicativity has a price, and it is the side condition on ∀-elim. Let $id = \lambda(P{:}\mathrm{Sort}\ 0).\ \lambda(h{:}P).\ h$, a proof of $\forall P.\ P \to P$. Logic lets us instantiate this theorem with itself to get $(\forall P.\ P \to P) \to (\forall P.\ P \to P)$, but the corresponding application is ill typed:

$$id : \Pi(P{:}\mathrm{Sort}\ 0).\ P \to P \qquad \text{and} \qquad \Pi(P{:}\mathrm{Sort}\ 0).\ P \to P : \mathrm{Sort}\ 1, \quad \text{so} \quad id\ (\Pi(P{:}\mathrm{Sort}\ 0).\ P \to P) \text{ is rejected b}$$

Higher levels are what make up for this. The same proof can be written at Sort 1, as $\lambda(P{:}\mathrm{Sort}\ 1).\ \lambda(h{:}P).\ h$, and that version does accept $\Pi(P{:}\mathrm{Sort}\ 0).\ P \to P$. Each level holds the level beneath it, and in practice a few levels suffice; higher levels appear when one quantifies over large collections, such as the category of all small categories.

7.5 Comparison with Lean

In Lean [5], Sort 0 is Prop and Sort $(u+1)$ is Type u. Prop has two extra features that our Sort 0 does not: *proof irrelevance* (any two proofs of the same proposition are equal) and *impredicativity* ($\forall\ (P : \mathrm{Prop}),\ P \to P$ is again in Prop, not one level up). Impredicativity removes the side condition on ∀-elim for propositions, and it stays consistent because it is confined to the bottom universe; it is Type : Type, not impredicativity by itself, that leads to Girard's paradox. Lean also has *universe polymorphism*, so definitions can be stated once for all levels u. In our system every level is a concrete number and every level follows the same rules.

Table 1. Design choices for universes.

Design	What can be quantified over	Drawback
No universes	values only	cannot say "for every proposition"
Type : Type	everything	inconsistent
Two sorts $* : \Box$	$*$ only	$\Box$ itself has no type
Sort u : Sort $(u+1)$	any level	levels must be tracked

8 A TYPE CHECKING ALGORITHM

The rules in Figure 2 define which judgments hold, but they are not yet an algorithm, because T-Conv can be applied at any point. The standard solution splits type checking into two mutually recursive functions [1]: Infer computes the type of a term, and Check verifies a term against an expected type. Conversion is used only where two types must be compared: in Check, and when a type must have a particular shape (a Π or a Sort).

Some remarks on Algorithm 1:

- Each case corresponds to exactly one rule of Figure 2, so the algorithm is *syntax-directed*. Finding a proof may be hard, but checking a given proof is mechanical.
- InferSort implements the premises of the form $\Gamma \vdash A : \mathrm{Sort}\ u$. It is also how the kernel decides whether a term is a type at all: A is a type exactly when its type normalizes to a universe. A λ-abstraction can never be a type, because its inferred type is always a Π.
- Freshen renames a binder whose name is already in the context, implementing the side condition of C-Ext.

Algorithm 1 Type inference and checking.

```
 1: function Infer(Γ, t)
 2:     if t = Sort u then return Sort (u + 1)
 3:     else if t = x then
 4:         if (x : A) ∈ Γ then return A
 5:         else fail "unbound variable"
 6:     else if t = Π(x:A). B then
 7:         u ← InferSort(Γ, A)
 8:         (x′, B′) ← Freshen(Γ, x, B)
 9:         v ← InferSort((Γ, x′ : A), B′)
10:         return Sort (max(u, v))
11:     else if t = λ(x:A). s then
12:         InferSort(Γ, A)
13:         (x′, s′) ← Freshen(Γ, x, s)
14:         B ← Infer((Γ, x′ : A), s′)
15:         return Π(x′:A). B
16:     else if t = f a then
17:         T ← Normalize(Infer(Γ, f))
18:         if T = Π(x:A). B then
19:             Check(Γ, a, A)
20:             return B[x ↦ a]
21:         else fail "not a function"

22: function Check(Γ, t, T)
23:     T′ ← Infer(Γ, t)
24:     if Normalize(T′) =α Normalize(T) then return
25:     else fail "type mismatch"

26: function InferSort(Γ, A)
27:     T ← Normalize(Infer(Γ, A))
28:     if T = Sort u then return u
29:     else fail "expected a type"

30: function Freshen(Γ, x, b)
31:     if x ∈ dom(Γ) then
32:         choose x′ ∉ dom(Γ) ∪ FV(b)
33:         return (x′, b[x ↦ x′])
34:     else return (x, b)
```

- Starting from the empty context, every context the algorithm builds is well formed, because each assumption is added only after InferSort succeeds. A caller that starts from a non-empty context must check that context separately.
- Termination depends on Normalize terminating, which holds for well-typed terms by strong normalization (§5).

9 WORKED EXAMPLES

The identity proof. Let $\Gamma_1 = P : \mathrm{Sort}\ 0$ and $\Gamma_2 = \Gamma_1, h : P$. Omitting the well-formedness premises $\vdash \Gamma$ ok, the derivation of the opening example is

$$\dfrac{\dfrac{}{\cdot \vdash \mathrm{Sort}\ 0 : \mathrm{Sort}\ 1}\ \textsc{T-Sort} \qquad \dfrac{\dfrac{\dfrac{}{\Gamma_1 \vdash P : \mathrm{Sort}\ 0}\ \textsc{T-Var} \qquad \dfrac{}{\Gamma_2 \vdash h : P}\ \textsc{T-Var}}{\Gamma_1 \vdash \lambda(h{:}P).\, h : P \to P}\ \textsc{T-Lam}}{\cdot \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h : \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P}\ \textsc{T-Lam}$$

Algorithm 1 produces the type $\Pi(P{:}\mathrm{Sort}\ 0).\ \Pi(h{:}P).\ P$. It is α-equivalent to the stated type, because $P \to P$ abbreviates $\Pi(_{:}P).\ P$ and the names h and _ do not occur in the body.

A wrong proof is rejected. For $\lambda(P{:}\mathrm{Sort}\ 0).\ \lambda(h{:}P).\ P$, the body P has type Sort 0, not P, so Check reports a type mismatch.

Using a theorem. Let $id = \lambda(P{:}\mathrm{Sort}\ 0).\ \lambda(h{:}P).\ h$ and $\Gamma = Q : \mathrm{Sort}\ 0$. By T-App,

$$\frac{\Gamma \vdash id : \Pi(P{:}\mathrm{Sort}\ 0).\ P \to P \qquad \Gamma \vdash Q : \mathrm{Sort}\ 0}{\Gamma \vdash id\ Q : (P \to P)[P \mapsto Q]}\ \text{T-App}$$

and $(P \to P)[P \mapsto Q] = Q \to Q$. Applying a proof of a universal statement to a particular proposition yields a proof about that proposition, and substitution computes what is being proved.

An exercise. Prove $\Pi(P{:}\mathrm{Sort}\ 0).\ \Pi(Q{:}\mathrm{Sort}\ 0).\ P \to Q \to P$. One answer is

$$\lambda(P{:}\mathrm{Sort}\ 0).\ \lambda(Q{:}\mathrm{Sort}\ 0).\ \lambda(p{:}P).\ \lambda(q{:}Q).\ p.$$

The proof never uses q; this system allows hypotheses to go unused.

10 A PITFALL: VARIABLE CAPTURE IN CONTEXTS

It is tempting to drop the side condition $x \notin \mathrm{dom}(\Gamma)$ and let a new assumption simply shadow an old one, as most programming languages do. Doing so makes the kernel unsound. Consider

$$t = \lambda(x{:}\mathrm{Sort}\ 0).\ \lambda(y{:}x).\ \lambda(x{:}\mathrm{Sort}\ 0).\ y.$$

The variable y has the *outer* x as its type, and the innermost λ ignores its argument. So the correct type of t is

$$\Pi(x{:}\mathrm{Sort}\ 0).\ \Pi(y{:}x).\ \Pi(x'{:}\mathrm{Sort}\ 0).\ x.$$

A checker without Freshen looks up y in the context $x : \mathrm{Sort}\ 0, y : x, x : \mathrm{Sort}\ 0$ and returns its type, the bare name x. At that point x refers to the inner binder, and the checker returns

$$\Pi(x{:}\mathrm{Sort}\ 0).\ \Pi(y{:}x).\ \Pi(x{:}\mathrm{Sort}\ 0).\ x,$$

which says "from any element of any type, produce an element of every type". Given $A : \mathrm{Sort}\ 0$ and $a : A$, the term $t\ A\ a$ then has type $\Pi(B{:}\mathrm{Sort}\ 0).\ B$, a proof of False. Any extension of the system with an inhabited base type, such as Nat with $0 : \mathrm{Nat}$, provides such A and a in the empty context.

The problem is the same as in substitution (§3): a name is moved into a scope where it means something else. It affects any representation that identifies variables by name, whether the context is a list or a hash map. There are three common remedies:

(1) Reject shadowing outright. This is the simplest option, but the reserved name _ used by $A \to B$ must be exempt and never added to the context.
(2) Rename on entry, as Freshen does.
(3) Represent bound variables by position (de Bruijn indices [2]) or use the *locally nameless* style, in which bound variables are indices and free variables carry unique identifiers. Lean 4 takes the latter approach.

A test suite should include t above and check that the kernel either rejects it or assigns it the correct type.

11 PROPERTIES

We state, without proof, the standard properties of this system. It is an instance of a Pure Type System with a predicative hierarchy, and the proofs can be found in Barendregt [1].

Theorem 2 (Uniqueness of types). *If $\Gamma \vdash t : A$ and $\Gamma \vdash t : B$, then $A \equiv B$.*

Theorem 3 (Type correctness). *If* $\Gamma \vdash t : A$*, then either* $A = \mathrm{Sort}\ u$ *for some* u*, or* $\Gamma \vdash A : \mathrm{Sort}\ u$ *for some* u*.*

Theorem 4 (Subject reduction). *If* $\Gamma \vdash t : A$ *and* $t \to_\beta t'$*, then* $\Gamma \vdash t' : A$*.*

Theorem 5 (Strong normalization). *If* $\Gamma \vdash t : A$*, then every sequence of* β*-reductions starting from* t *is finite.*

Together these give the two guarantees we want from a kernel. *Decidability*: Algorithm 1 terminates on every input, so whether a proof is correct can always be determined. *Consistency*: there is no closed term of type $\Pi(P{:}\mathrm{Sort}\ 0).\, P$. By strong normalization and subject reduction, such a term would have a normal form of the same type. That normal form must be $\lambda(P{:}\mathrm{Sort}\ 0).\, s$ with $P : \mathrm{Sort}\ 0 \vdash s : P$ and s normal. A normal term is a universe, a Π, a λ, or a variable applied to arguments, and none of these can have type P in that context: the only variable available is P itself, whose type is $\mathrm{Sort}\ 0$, not P.

REFERENCES

[1] H. P. Barendregt. Lambda calculi with types. In *Handbook of Logic in Computer Science*, 1992.
[2] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation. *Indagationes Mathematicae*, 1972.
[3] J.-Y. Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. PhD thesis, 1972.
[4] A. J. C. Hurkens. A simplification of Girard's paradox. *TLCA*, 1995.
[5] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADE*, 2021.